

9.2 RLHF

Three stages. Only PPO samples the current policy. $\beta=0$ is reward hacking.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

RLHF (InstructGPT / ChatGPT-style) is a three-stage stack: SFT, a reward model, then a **policy** optimized with RL (usually **PPO**) so sampled replies score high under the RM without wandering too far from SFT.

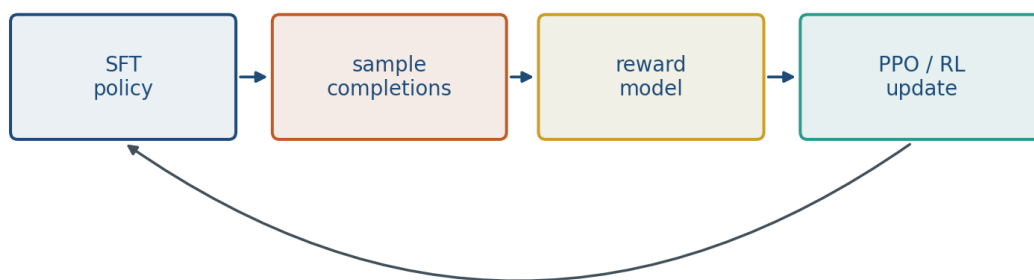
First time this method appears. RLHF is three stages: SFT, reward model, then **PPO** on the policy with a KL leash.

What. After SFT and an RM, you sample from the **current** policy, score with r , update to raise reward minus KL to a frozen reference. **Why.** SFT does not explore. Best-of- n uses r only at decode. RLHF trains π to look high-reward **as it samples**. **Architecture.** Policy π_θ , reference π_{ref} (usually SFT), RM r (frozen in the PPO stage), optional critic. **How.** PPO clip + advantage. $\beta = 0$ (no KL) is reward hacking: the policy exploits r . **Formula.** $\max_{\pi} \mathbb{E}[r(x, y)] - \beta \text{KL}(\pi(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$. **Tradeoffs.** + Can beat SFT on preference evals. – Unstable, expensive, RM misspecification; people look to DPO (next note) to skip PPO.

1. The loop

1. **SFT** on instruction pairs (note **8.1**). Call this π_{SFT} .
2. **RM** r_ϕ on chosen/rejected pairs (note **9.1**).
3. **PPO**: sample $y \sim \pi_\theta(\cdot | x)$, score with r_ϕ , update π_θ .

RLHF: climb the reward, stay near the SFT policy (KL penalty)



PPO is on-policy: you need fresh rollouts from the current π_θ . That is expensive (generate, reward, critic). The **critic** (value head) estimates advantages so the policy gradient has lower variance. You do not need to derive PPO in this course; you need the picture and the KL term.

Only stage 3 needs samples from the **current** policy. Stages 1–2 use a fixed dataset.

2. The KL penalty

Without a constraint, π_θ **hacks** r_ϕ : long, flattering, or degenerate text that the RM likes. The usual objective looks like

$$\mathbb{E}_{x, y \sim \pi_\theta} [r_\phi(x, y) - \beta \text{KL}(\pi_\theta(\cdot | x) \| \pi_{\text{ref}}(\cdot | x))],$$

with $\pi_{\text{ref}} = \pi_{\text{SFT}}$ frozen. β is the knob: large β stays near SFT; small β chases reward. In code this is often a per-token penalty $\beta \log \pi_\theta - \beta \log \pi_{\text{ref}}$.

If $\beta = 0$, there is no leash. Expect reward to rise while KL explodes and the text becomes a caricature of the raters.

3. Why people look past PPO

It works at company scale and is finicky in a seminar: sampling, KL, reward scale, and a moving policy. **DPO** (next note) skips the RM and the loop. RLHF is still the conceptual parent: preferences $\rightarrow$ a scalar of “better” $\rightarrow$ a policy that maximizes it with a leash.

For a project, running full PPO on an LLM is usually out of scope. Best-of- n with a tiny RM, or DPO on a small classifier, is enough to show you understood the stack.

On-policy means last week’s samples are stale: π_θ moved, so you generate again. That is the cost people are avoiding with DPO. Keep the three-box picture even if you never run PPO.

4. Teaching this note

About **35 minutes** at the board: three boxes (SFT, RM, PPO), then write the KL objective and set $\beta = 0$ vs large β . Play the same Karpathy **RLHF** window **21:05–25:43** (and a minute after if he is still on comparisons). DPO is next; do not derive PPO.

5. Worked example

Stage check: a batch of instruction pairs $\rightarrow$ SFT. A batch of (y_w, y_l) $\rightarrow$ RM. A batch of **new** samples from $\pi_\theta \rightarrow$ PPO. If your code loads a CSV of completions and never calls `generate` inside the PPO loop, it is not on-policy RLHF.

KL cartoon: two distributions on a 2-word vocab $\{a, b\}$. $\pi_{\text{ref}} = (0.7, 0.3)$, $\pi_\theta = (0.99, 0.01)$.

$$\text{KL}(\pi_\theta \| \pi_{\text{ref}}) = 0.99 \log \frac{0.99}{0.7} + 0.01 \log \frac{0.01}{0.3} \approx 0.99 \cdot 0.347 + 0.01 \cdot (-3.40) \approx 0.31.$$

Penalty $\beta \cdot 0.31$. If $\beta = 0.1$, you subtract 0.031 from reward; if $\beta = 0$, you subtract nothing and π_θ can spike.

 A two-word spike: mass on <math xmlns=

a goes 0.7 $\rightarrow$ 0.99" src="graphics/9.2-rlhf/kl-numeric.png" />

The three boxes are SFT, RM, and RL. **Best-of- n** is already a strong baseline in AlpacaFarm-style studies. PPO still exists because on-policy sampling can find replies that were never in the pair file. InstructGPT is this stack at company scale; a seminar project usually stops at best-of- n or DPO.

Why $\pi_{\text{ref}} = \pi_{\text{SFT}}$ not the raw pretrained LM: the RM was trained on SFT-style answers. Leashing to the base model pulls you back to web-text, not to the assistant you already paid for.

6. Where students get stuck

- Drawing RLHF as one loss. It is three stages; only the last is RL.
 - Setting $\beta = 0$ “to learn faster” and then being surprised by reward hacking.
 - Using the pretrained base as π_{ref} after SFT already moved the policy.
-

7. Video

[Karpathy — Intro to Large Language Models](#). Play **21:05–25:43** (RLHF, comparison labels). Pause on the three-stage story: write, compare, then a loop that uses those comparisons. This clip will not derive PPO; the board does the KL leash.

8. Practice

1. Which of the three stages needs *samples from the current policy*, not a fixed dataset?
2. If $\beta = 0$, what failure mode should you expect?
3. Why is π_{ref} the SFT model rather than the raw pretrained LM?
4. $\pi_\theta = (0.8, 0.2)$, $\pi_{\text{ref}} = (0.8, 0.2)$. What is $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$, and what does that say about the leash at this instant?
5. Reward $r = 5$, $\beta = 2$, $\text{KL} = 1.5$. What scalar is the objective $r - \beta \text{KL}$? If you raise β to 4, who wins: the RM or the reference?